

# Ruby

POUR DÉBUTANTS

Les bases du langage pour  
bien démarrer en programmation



DES EXEMPLES CONCRETS  
DES EXERCICES PRATIQUES  
UN PROJET PAS À PAS



De zéro  
à vos premiers  
programmes

FORMATION RUBY - LES BASES

## Ruby

Apprendre le langage elegant de Matz, pas  
à pas.

DanielCraft - Livre debutant - Pack Web

# Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

## 1. [C'est quoi Ruby ?](#)

- [Le langage en une phrase](#)
- [Pourquoi apprendre Ruby ?](#)
- [A retenir](#)

## 2. [Installer Ruby](#)

- [Windows](#)
- [Mac / Linux](#)
- [Editeur](#)
- [A retenir](#)

## 3. [Premier programme](#)

- [Affichage formate](#)
- [Commentaires](#)
- [A retenir](#)

## 4. [Les variables](#)

- [Conventions](#)
- [A retenir](#)

## 5. [Les types](#)

- [Verifier un type](#)
- [Conversion](#)
- [A retenir](#)

## 6. [Les conditions](#)

- [Modificateur if](#)
- [case \(switch\)](#)
- [A retenir](#)

## 7. [Les boucles](#)

- [each \(idiome principal\)](#)
- [while / until](#)
- [times](#)
- [break et next](#)
- [A retenir](#)

## 8. [Les methodes](#)

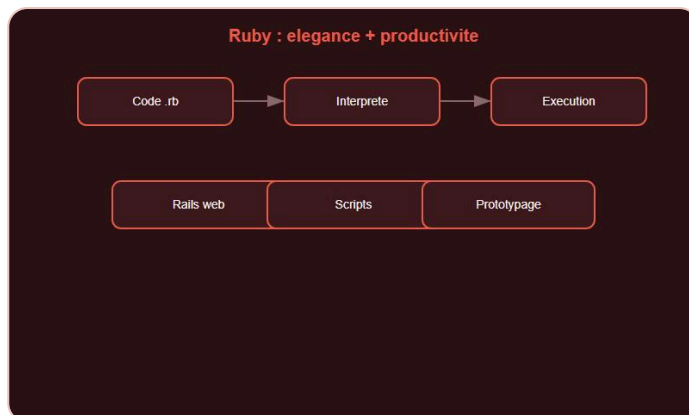
- [Parametres par default](#)
- [Blocs](#)
- [A retenir](#)

## 9. [Symboles et hashes](#)

- [Symboles](#)
- [Hashes](#)
- [Syntaxe moderne](#)
- [A retenir](#)

10. Classes et objets
  - Attributs
  - A retenir
11. Modules et heritage
  - Heritage
  - Modules (mixins)
  - A retenir
12. Gerer les erreurs
  - Lever une erreur
  - A retenir
13. Mini-projet : gestionnaire de taches CLI
  - A retenir
14. Ce qu'il faut retenir
15. Atelier : variables
16. Atelier : methodes
17. Atelier : classes
18. Erreurs classiques
  - 1. Oublier end
  - 2. Confondre symbole et string
  - 3. Modifier un array en iteration
  - 4. nil implicite
19. Bonnes pratiques
  - A retenir
20. Quiz Ruby - Les bases
21. Bravo !
  - La suite

# C'est quoi Ruby ?



Ruby : elegance et productivite.



Couverture Ruby.

## Le langage en une phrase

Ruby est un langage orienté objet, conçu pour être agréable à écrire et à lire. Matz (Yukihiro Matsumoto) l'a créé en 1995 avec la devise : rendre les programmeurs heureux.

## Pourquoi apprendre Ruby ?

Ruby alimente Ruby on Rails, l'un des frameworks web les plus influents. Il reste populaire pour le prototypage rapide, les scripts et le backend web.

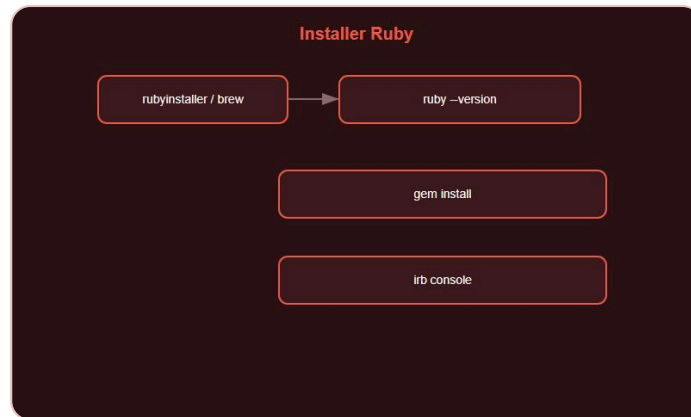
> **Astuce DanielCraft** - Ruby te apprend à écrire du code élégant. Même si tu codes ailleurs, cette élégance reste.

```
nom = "Lea"
puts "Salut #{nom}, bienvenue en Ruby !"
```

## A retenir

- Ruby = elegance + productivite.
- Tout est objet.
- Idéal pour le web (Rails) et les scripts.

## Installer Ruby



Ruby + gem + irb.

### Windows

1. 1. Télécharge **Ruby+Devkit** sur [rubyinstaller.org](http://rubyinstaller.org).
2. 2. Lance l'installateur et coche "Add to PATH".
3. 3. Vérifie : `ruby --version`.

### Mac / Linux

```
# Mac
brew install ruby

# Linux
sudo apt install ruby-full
```

### Editeur

VS Code + extension Ruby, ou RubyMine (JetBrains).

> **Astuce DanielCraft** - `gem` est le gestionnaire de paquets Ruby (comme npm ou pip).

### A retenir

- `ruby --version` pour vérifier.
- `gem install nom_gem` pour les bibliothèques.

## Premier programme



puts et interpolation #{ }.

```
puts "Bonjour le monde !"
```

### Affichage formate

```
nom = "Sam"  
age = 28  
puts "Je suis #{nom}, #{age} ans."
```

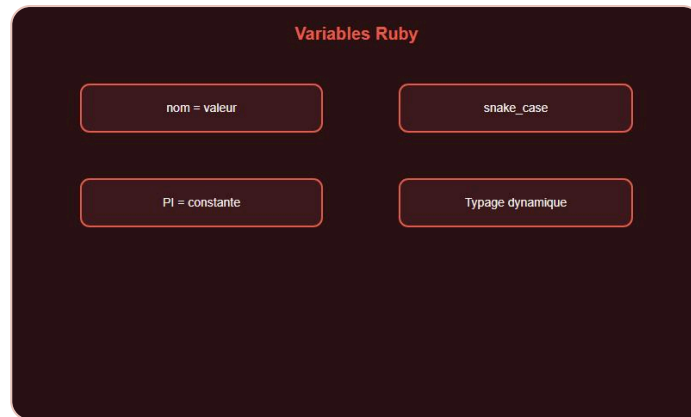
### Commentaires

```
# Ligne unique  
=begin  
Bloc  
=end
```

### A retenir

- `puts` affiche avec retour ligne.
- Interpolation avec `#{expression}`.

## Les variables



Assignation et constantes.

```
nom = "Max"      # Reassignable  
PI = 3.14        # Constante (MAJUSCULES)
```

Ruby n'a pas de `let` / `var` : une variable est assignee directement.

### Conventions

- snake\_case pour variables et methodes.
- PascalCase pour classes et modules.
- SCREAMING\_SNAKE\_CASE pour constantes.

> **Astuce DanielCraft** - Une constante Ruby peut etre reassignee (avec un avertissement). C'est une convention, pas une protection stricte.

### A retenir

- Assignation directe : `nom = valeur`.
- Constantes en MAJUSCULES.
- snake\_case partout.

# Les types



Typage dynamique et nil.

Ruby est dynamique : pas de declaration de type.

| Type    | Exemple                 |
|---------|-------------------------|
| Integer | 42                      |
| Float   | 3.14                    |
| String  | "Bonjour"               |
| Boolean | true , false            |
| nil     | nil (absence de valeur) |

## Verifier un type

```
42.class      # Integer
"texte".is_a?(String) # true
```

## Conversion

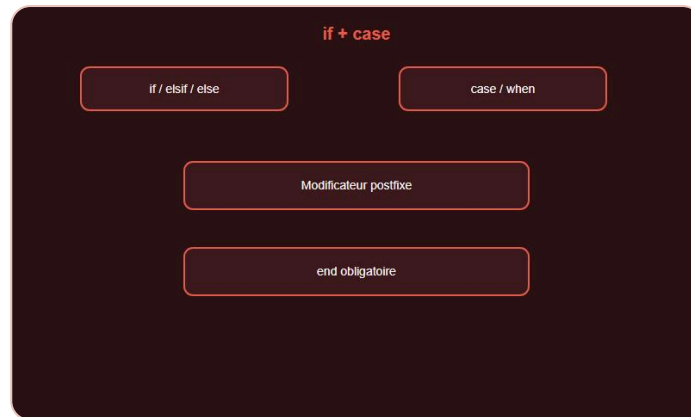
```
"42".to_i    # 42
42.to_s      # "42"
```

## A retenir

- Typage dynamique.
- `nil` = absence de valeur.
- `.to_i`, `.to_s`, `.to_f` pour convertir.



## Les conditions



if et case/when.

```
if age >= 18
  puts "Majeur"
elsif age >= 12
  puts "Adolescent"
else
  puts "Enfant"
end
```

### Modificateur if

```
puts "Majeur" if age >= 18
```

### case (switch)

```
case jour
when "lundi", "mardi"
  puts "Debut de semaine"
when "vendredi"
  puts "Weekend proche"
else
  puts "Autre"
end
```

### A retenir

- `if / elsif / else / end`.
- `case / when / else / end`.
- Modificateurs postfixe possibles.

# Les boucles



.each, while, times.

## each (idiome principal)

```
(0..4).each { |i| puts i }

fruits = ["pomme", "banane"]
fruits.each { |f| puts f }
```

## while / until

```
n = 0
while n < 5
  puts n
  n += 1
end
```

## times

```
5.times { |i| puts i }
```

## break et next

```
(0..10).each do |i|
  break if i == 5
  next if i.even?
  puts i
end
```

> **Astuce DanielCraft** - Prefere `.each` aux boucles `for`. C'est l'idiome Ruby.

## A retenir

- `.each` pour parcourir.
- `(0..n)` pour les ranges inclusives.
- `break` / `next` comme continue.

## Les methodes



Methodes, blocs et yield.

```
def saluer(prenom)
  puts "Bonjour #{prenom} !"
end

def additionner(a, b)
  a + b
end
```

La dernière expression est retournée automatiquement (pas de `return` obligatoire).

### Parametres par défaut

```
def saluer(prenom, message = "Bonjour")
  puts "#{message} #{prenom} !"
end
```

### Blocs

```
def repeter(n)
  n.times { yield }
end

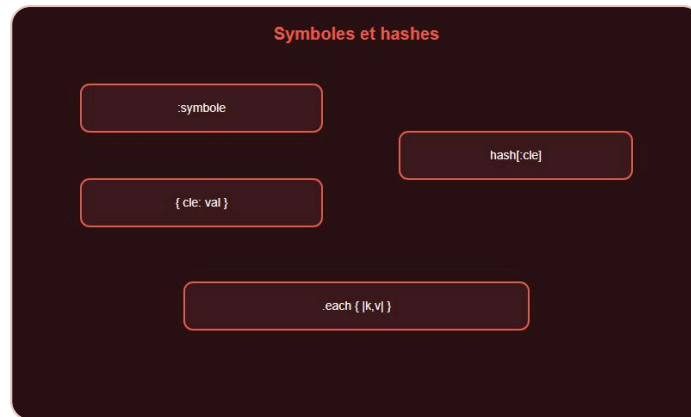
repeter(3) { puts "Ruby !" }
```

> **Astuce DanielCraft** - Les blocs `{ }` ou `do..end` sont partout en Ruby. C'est une force du langage.

### A retenir

- `def nom(params) / end`.
- Retour implicite (dernière expression).
- Blocs avec `yield`.

## Symboles et hashes



Symboles `:cle` et hashes.

### Symboles

```
statut = :actif
puts statut.class # Symbol
```

Un symbole est immutable et unique en memoire. Ideal pour les cles.

### Hashes

```
ages = { lea: 28, sam: 22, nora: 30 }
puts ages[:lea] # 28
ages[:max] = 25
```

### Syntaxe moderne

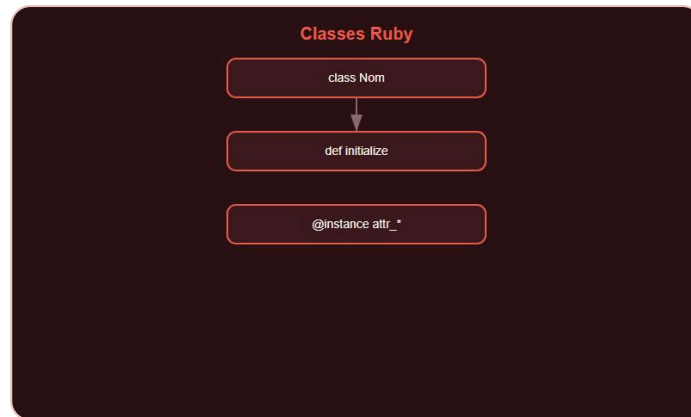
```
personne = { nom: "Lea", ville: "Paris" }
personne.each { |cle, val| puts "#{cle} = #{val}" }
```

> **Astuce DanielCraft** - `:{cle}` cree un symbole. Les hashes sont omnipresents en Ruby (surtout Rails).

### A retenir

- `:symbole` = identifiant immutable.
- `{ cle: valeur }` = hash.
- Acces avec `[:cle]`.

# Classes et objets



Classes et @instance.

```

class Animal
  def initialize(nom, age)
    @nom = nom
    @age = age
  end

  def se_presenter
    "Je suis #{@nom}, #{@age} ans."
  end
end

chat = Animal.new("Felix", 3)
puts chat.se_presenter
  
```

## Attributs

```

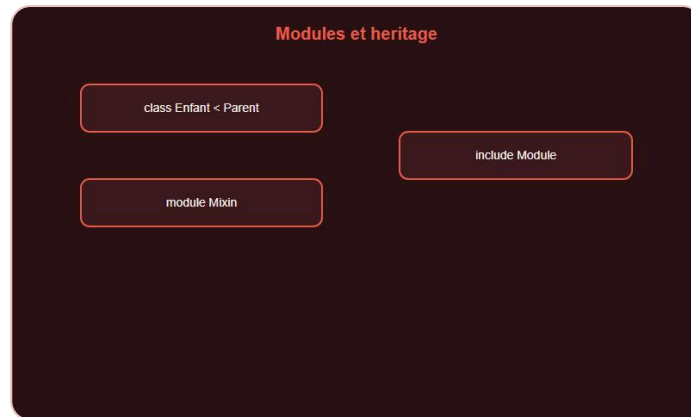
class Produit
  attr_reader :nom
  attr_accessor :prix

  def initialize(nom, prix)
    @nom = nom
    @prix = prix
  end
end
  
```

## A retenir

- `class` / `def initialize` / `end`.
- `@variable` = variable d'instance.
- `attr_reader`, `attr_accessor`.

## Modules et héritage



Héritage et mixins.

### Héritage

```
class Chien < Animal
  def parler
    "Wouf !"
  end
end
```

### Modules (mixins)

```
module Affichable
  def afficher
    puts to_s
  end
end

class Produit
  include Affichable
end
```

### A retenir

- `< Parent` pour hériter.
- `module` + `include` pour partager du comportement.
- Ruby : héritage simple + mixins.

## Gérer les erreurs



begin / rescue / raise.

```
begin
  resultat = 10 / 0
rescue ZeroDivisionError => e
  puts "Erreur : #{e.message}"
ensure
  puts "Toujours execute"
end
```

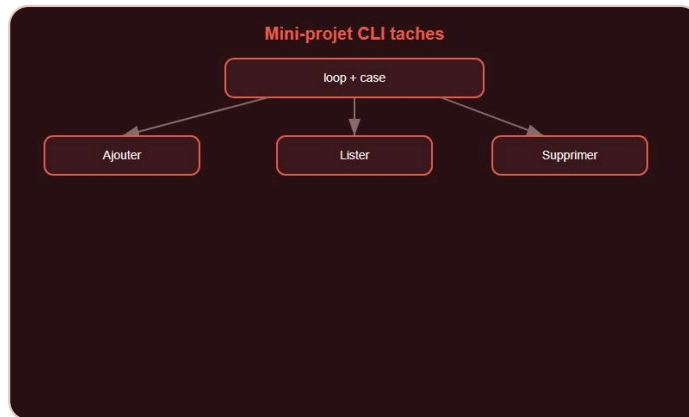
### Lever une erreur

```
def retirer(solde, montant)
  raise "Solde insuffisant" if montant > solde
  solde - montant
end
```

### A retenir

- `begin / rescue / ensure / end`.
- `raise` pour lever une exception.
- `rescue => e` pour attraper.

## Mini-projet : gestionnaire de tâches CLI



Mini-projet : gestionnaire de tâches.

```

taches = []

loop do
  puts "\n1. Ajouter 2. Lister 3. Supprimer 4. Quitter"
  choix = gets.chomp

  case choix
  when "1"
    print "Tache : "
    tache = gets.chomp
    taches << tache unless tache.empty?
    puts "Ajoutée."
  when "2"
    if taches.empty?
      puts "Aucune tache."
    else
      taches.each_with_index { |t, i| puts "  #{i + 1}. #{t}" }
    end
  when "3"
    print "Numero : "
    idx = gets.to_i - 1
    taches.delete_at(idx) if idx >= 0 && idx < taches.size
  when "4"
    puts "A bientôt !"
    break
  end
end
end
  
```

### A retenir

- Array + case + gets = CLI complet.
- `loop do` / `break` pour le menu.



# Ce qu'il faut retenir



Carte des notions Ruby.

| Concept    | Resume                          |
|------------|---------------------------------|
| Variables  | Assignation directe, snake_case |
| Types      | Dynamiques, nil                 |
| Conditions | if, case/when                   |
| Boucles    | .each, while, times             |
| Methodes   | def, blocs, yield               |
| Symboles   | :nom immutable                  |
| Hashes     | { cle: val }                    |
| Classes    | @instance, attr_*               |
| Modules    | include mixin                   |
| Erreurs    | begin/rescue/raise              |

> **Astuce DanielCraft** - Ruby recompense l'elegance. Ecris peu, exprime beaucoup.

## Atelier : variables



Atelier : variables.

```
prenom = "Sam"
age = 22
puts "Je suis #{prenom}, #{age} ans."

celsius = 37.0
fahrenheit = celsius * 9 / 5 + 32
puts "#{celsius}°C = #{fahrenheit}°F"
```

## Atelier : methodes



Atelier : methodes.

```
def saluer(nom, heure)
  if heure < 12
    "Bonjour #{nom} !"
  elsif heure < 18
    "Bon apres-midi #{nom} !"
  else
    "Bonsoir #{nom} !"
  end
end

def moyenne(notes)
  return 0 if notes.empty?
  notes.sum.to_f / notes.size
end
```

## Atelier : classes



Atelier : classes.

```
class Produit
  attr_reader :nom
  attr_accessor :prix

  def initialize(nom, prix)
    @nom = nom
    @prix = prix
  end

  def afficher
    "#{@nom} - #{@prix.round(2)} EUR"
  end
end
```

# Erreurs classiques



Pieges classiques Ruby.

## 1. Oublier end

Chaque `def`, `class`, `if`, `do` nécessite un `end`.

## 2. Confondre symbole et string

```
hash = { nom: "Lea" }
hash["nom"] # nil !
hash[:nom]  # "Lea"
```

## 3. Modifier un array en iteration

```
arr = [1, 2, 3]
arr.each { |x| arr.delete(x) } # Comportement inattendu
```

## 4. nil implicite

```
resultat = hash[:absent]
resultat.upcase # NoMethodError sur nil
```

Utilise `&.` (safe navigation) ou vérifie `nil`.

> **Piege** - Ruby est permissif. Teste souvent en console (`irb`).

## Bonnes pratiques



RuboCop et bonnes pratiques.

- **RuboCop** : linter et formateur standard.
- Prefere `.each` a `for`.
- Methodes courtes, une responsabilite.
- Noms expressifs : `utilisateur_actif?` (le `?` indique un boolean).
- Tests avec **RSpec** ou Minitest.

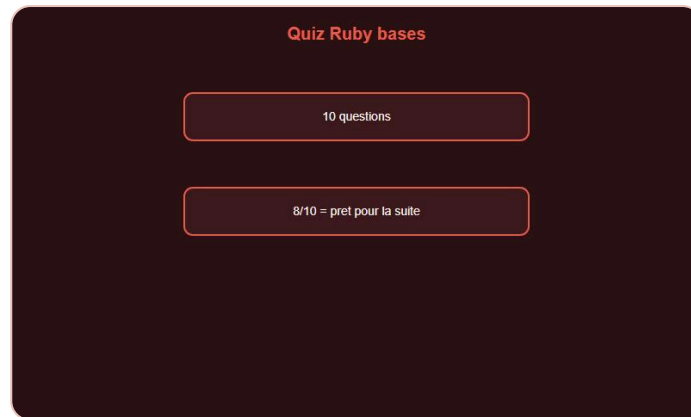
```
def utilisateur_majeur?(age)
  age >= 18
end
```

### A retenir

- Code idiomatique Ruby, pas du Java traduit.
- RuboCop + tests = qualite.

## QUIZ

### Quiz Ruby - Les bases



Quiz Ruby bases.

- Q1. Ruby est : A) compile B) dynamique C) sans objets D) sans nil → **B**
- Q2. Constante : A) const x B) X en majuscules C) final D) let → **B**
- Q3. :actif est : A) String B) Symbol C) Hash D) Array → **B**
- Q4. Hash acces : A) hash("cle") B) hash[:cle] C) hash.cle D) hash->cle → **B**
- Q5. Retour implicite : A) Non B) Derniere expression C) return obligatoire D) nil toujours → **B**
- Q6. Framework web : A) Django B) Rails C) Laravel D) Spring → **B**
- Q7. .each remplace : A) while surtout B) for en idiome C) case D) raise → **B**
- Q8. nil en Ruby = A) 0 B) false C) absence de valeur D) erreur → **C**
- Q9. @nom = A) globale B) locale C) instance D) classe → **C**
- Q10. include Module = A) heritage B) mixin C) import D) require → **B**

FINAL

**Bravo !**



Bravo. Tu codes en Ruby.

Tu as terminé **Ruby - Les bases**. Tu maîtrises variables, méthodes, blocs, symboles, hash, classes, modules et gestion d'erreurs.

### La suite

- **Ruby - Intermédiaire** : Rails, gems, tests RSpec, métaprogrammation légère.

> **Astuce DanielCraft** - Ouvre `irb` et expérimente. Ruby se apprend en jouant.

Tu as les bases. Maintenant, code avec joie.

Tu as les briques. Maintenant, code avec joie.